BigQuery Fundamentals

A Hands-On Guide Using a Synthetic Indian Employee Dataset

This guide walks through every core BigQuery concept — loading data, basic querying, aggregation, joins, views, materialized views, partitioning, and clustering — using three small, easy-to-read CSV files built specifically for this purpose: **employees.csv**, **departments.csv**, and **attendance_monthly.csv**. It's meant to be read alongside the CSVs, either as a print-friendly companion to a live hands-on notebook, or as a standalone teaching reference.

Problem statement: A small Indian technology company ('IndexCorp') wants to move its HR data into BigQuery to answer basic workforce questions — headcount by department, salary distribution by gender and state, attendance trends over time — and eventually wants those queries to run fast and cheaply even as the attendance history grows to millions of rows. This guide builds up the exact BigQuery techniques needed to do that, one concept at a time.

1. The Dataset

Three CSV files, designed to work together as a small but realistic HR data model — one dimension table for departments, one dimension/fact hybrid for employees, and one large fact table for monthly attendance, which is what makes the partitioning and clustering sections (Sections 8-9) meaningful rather than abstract.

1.1 departments.csv — 8 rows (dimension table)

One row per department. Small, slowly-changing — a classic dimension table.

Column	Type	Description
department_id	INTEGER	Unique department identifier (101-108)
department_name	STRING	e.g. Engineering, Sales, Finance
location	STRING	City where the department is based
department_head	STRING	Name of the department's head

1.2 employees.csv — 100 rows (dimension/fact hybrid)

One row per employee. The main table most basic queries and joins will use.

Column	Type	Description
employee_id	STRING	Unique ID, e.g. EMP1000
first_name	STRING	Employee's first name
last_name	STRING	Employee's last name
gender	STRING	Male / Female
state	STRING	Indian state of residence
city	STRING	City corresponding to the state
department_id	INTEGER	Foreign key to departments.department_id
designation	STRING	Job title, e.g. Software Engineer, Manager
date_of_joining	DATE	YYYY-MM-DD
monthly_salary_inr	FLOAT	Monthly salary in Indian Rupees
email	STRING	Employee's email address

1.3 attendance_monthly.csv — 1,200 rows (large fact table)

One row per employee, per month, for all of 2025 (100 employees × 12 months). This is the table we'll use for partitioning and clustering later, since it's the only one with a real time dimension and enough rows to make pruning meaningfully visible.

Column	Type	Description
employee_id	STRING	Foreign key to employees.employee_id

Column	Type	Description
department_id	INTEGER	Denormalized for convenient clustering/filtering
state	STRING	Denormalized for convenient clustering/filtering
month_date	DATE	First day of the month, e.g. 2025-03-01
days_present	INTEGER	Days the employee was present that month
leaves_taken	INTEGER	Days of leave taken that month
performance_rating	FLOAT	Self-reported monthly rating, 1.0-5.0

Why denormalize department_id and state into attendance_monthly? In a real warehouse, you'd often join back to employees for these. We include them directly here so Sections 8-9 can demonstrate partitioning and clustering on this table without requiring a join in every single example query — a common, deliberate trade-off in analytics tables (trading a little redundancy for simpler, cheaper downstream queries).

2. Loading the CSVs Into BigQuery

Before any SQL, the three CSVs need to exist as BigQuery tables. The **bq load** command does this directly from a local file or a Cloud Storage path, with schema auto-detection so you don't have to hand-write a schema for a simple case like this.

2.1 Create a Dataset

```
bq mk --dataset --location=US YOUR_PROJECT_ID:hr_analytics
```

2.2 Load Each CSV

```
bq load --source_format=CSV --skip_leading_rows=1 --autodetect \
  YOUR_PROJECT_ID:hr_analytics.departments \
  departments.csv

bq load --source_format=CSV --skip_leading_rows=1 --autodetect \
  YOUR_PROJECT_ID:hr_analytics.employees \
  employees.csv

bq load --source_format=CSV --skip_leading_rows=1 --autodetect \
  YOUR_PROJECT_ID:hr_analytics.attendance_monthly \
  attendance_monthly.csv
```

On --autodetect: BigQuery scans a sample of rows to guess each column's type. This is convenient for small, clean files like these, but in a production pipeline it's safer to declare an explicit schema (via --schema=field:TYPE,field:TYPE,... or a JSON schema file) so a single malformed row can't silently change a column's inferred type.

Check it in the console: BigQuery > SQL workspace > Explorer → your project → hr_analytics → all three tables should now be listed. Click any one of them → the Schema tab shows exactly what --autodetect inferred, and the Preview tab shows the loaded rows without running a query.

3. Basic Queries — SELECT, WHERE, ORDER BY

The question we're answering: "Show me all employees in Karnataka earning over ■80,000 a month, highest paid first." The simplest possible shape of query: filter rows, sort them, look at the result.

```
SELECT
  employee_id, first_name, last_name, state, designation, monthly_salary_inr
FROM `hr_analytics.employees`
WHERE state = 'Karnataka' AND monthly_salary_inr > 80000
ORDER BY monthly_salary_inr DESC
```

Check it in the console: Paste this into the BigQuery SQL workspace and click Run. The Job information panel below the results grid shows Bytes processed — on a 100-row table this will be tiny (a few KB), which is worth contrasting later with the multi-row attendance table.

3.1 Aggregate Functions — COUNT, AVG, SUM

The question: "What's the average salary company-wide, and how many employees do we have?"

```
SELECT
  COUNT(*) AS total_employees,
  ROUND(AVG(monthly_salary_inr), 2) AS avg_salary,
  MIN(monthly_salary_inr) AS min_salary,
  MAX(monthly_salary_inr) AS max_salary
FROM `hr_analytics.employees`
```

3.2 GROUP BY — Headcount and Average Salary by State

The question: "Break that down by state." GROUP BY collapses all rows sharing the same state into a single output row, with the aggregate functions computed per group.

```
SELECT
  state,
  COUNT(*) AS headcount,
  ROUND(AVG(monthly_salary_inr), 2) AS avg_salary
FROM `hr_analytics.employees`
GROUP BY state
ORDER BY headcount DESC
```

Check it in the console: Run this, then click any column header in the results grid to sort interactively without re-running the query — a quick way to explore a small result set live.

3.3 A Window Function — Gender Pay Comparison Within Each Department

The question: "For each employee, how does their salary compare to their department's average?" A window function (AVG(...) OVER (PARTITION BY ...)) computes an aggregate *per group* while still returning one row per employee — unlike GROUP BY, which collapses rows.

```
SELECT
  e.employee_id, e.first_name, e.department_id, e.monthly_salary_inr,
  ROUND(AVG(e.monthly_salary_inr) OVER (PARTITION BY e.department_id), 2)
    AS dept_avg_salary,
  ROUND(e.monthly_salary_inr -
    AVG(e.monthly_salary_inr) OVER (PARTITION BY e.department_id), 2)
    AS diff_from_dept_avg
FROM `hr_analytics.employees` AS e
ORDER BY e.department_id, diff_from_dept_avg DESC
```

GROUP BY vs. window functions: GROUP BY answers "what's the average per department" (one row per department). A window function answers "what's the average per department, shown alongside every individual employee row" (one row per employee, still). Both are useful; the choice depends on whether you need to keep the individual rows visible.

4. Joins

The question: "Show me employee names alongside their department name and location — not just a department_id number." employees.csv only has department_id; the readable department_name lives in departments.csv. A join combines rows from both tables based on a matching key.

4.1 INNER JOIN — Employees With Department Details

```
SELECT
  e.employee_id, e.first_name, e.last_name,
  d.department_name, d.location
FROM `hr_analytics.employees` AS e
INNER JOIN `hr_analytics.departments` AS d
  ON e.department_id = d.department_id
LIMIT 20
```

INNER JOIN keeps only matches: if an employee's department_id didn't exist in the departments table (e.g. a typo, or a department that was since deleted), that employee would simply not appear in the result at all — silently dropped. Section 4.3 shows how to catch exactly this.

4.2 Aggregation Across a Join — Average Salary per Department

The question: "Which departments pay the most on average, and how many people are in each?" This combines a join with GROUP BY — a very common real-world pattern.

```
SELECT
  d.department_name, d.location,
  COUNT(e.employee_id) AS headcount,
  ROUND(AVG(e.monthly_salary_inr), 2) AS avg_salary
FROM `hr_analytics.departments` AS d
INNER JOIN `hr_analytics.employees` AS e
  ON d.department_id = e.department_id
GROUP BY d.department_name, d.location
ORDER BY avg_salary DESC
```

4.3 LEFT JOIN — Find Departments With Zero Employees

The question: "Do we have any departments with nobody assigned to them?" A LEFT JOIN keeps every row from the left table (departments) regardless of whether a match exists on the right (employees) — unmatched rows get NULL for every employees column. Filtering WHERE e.employee_id IS NULL isolates exactly the departments with no staff.

```
SELECT
  d.department_id, d.department_name
FROM `hr_analytics.departments` AS d
LEFT JOIN `hr_analytics.employees` AS e
  ON d.department_id = e.department_id
WHERE e.employee_id IS NULL
```

Check it in the console: Run each of these three in the SQL workspace. For the LEFT JOIN, try changing it back to INNER JOIN and re-running — the departments-with-no-staff row (if any) will silently disappear, a concrete demonstration of the exact difference between the two join types.

5. Views

The problem this solves: the department-summary join from Section 4.2 is useful, but re-typing that JOIN + GROUP BY every time an analyst wants this information is repetitive and error-prone. A VIEW stores the *query itself*, not its results — querying the view re-runs the underlying SELECT against the live base tables every time.

```
CREATE VIEW `hr_analytics.department_summary_view` AS
SELECT
  d.department_name, d.location,
  COUNT(e.employee_id) AS headcount,
  ROUND(AVG(e.monthly_salary_inr), 2) AS avg_salary
FROM `hr_analytics.departments` AS d
INNER JOIN `hr_analytics.employees` AS e
  ON d.department_id = e.department_id
GROUP BY d.department_name, d.location
```

Now analysts can just query the view directly, as if it were a normal table:

```
SELECT * FROM `hr_analytics.department_summary_view`
ORDER BY avg_salary DESC
```

Check it in the console: hr_analytics dataset in the Explorer → department_summary_view should now be listed with a small "view" icon. Click it → the Details tab shows Table type: VIEW and the exact SQL stored — confirming a view holds no data of its own, only a query definition.

Advantage vs. trade-off: A view is always 100% live (it can never show stale data) and costs nothing to store. But it gives *no performance benefit* — every query against it re-runs the full underlying join and aggregation from scratch, exactly as expensive as writing the raw SQL yourself. That's the problem Section 6 (Materialized Views) solves.

6. Materialized Views

The problem this solves: if department_summary_view were queried constantly by a live dashboard, re-computing the join and aggregation on every single refresh is wasteful. A MATERIALIZED VIEW actually *precomputes and stores* the result, then keeps it incrementally refreshed as the base tables change.

```
CREATE MATERIALIZED VIEW `hr_analytics.monthly_attendance_summary_mv` AS
SELECT
  a.month_date,
  a.department_id,
  ROUND(AVG(a.days_present), 2) AS avg_days_present,
  ROUND(AVG(a.performance_rating), 2) AS avg_performance_rating,
  COUNT(*) AS employee_month_records
FROM `hr_analytics.attendance_monthly` AS a
GROUP BY a.month_date, a.department_id
```

```
-- Query it exactly like a table
SELECT *
FROM `hr_analytics.monthly_attendance_summary_mv`
WHERE month_date = '2025-06-01'
ORDER BY avg_performance_rating DESC
```

Check it in the console: hr_analytics dataset → monthly_attendance_summary_mv should show a distinct "materialized view" icon. Its Details tab has a Refresh section (last refresh time, staleness) that a plain view's Details tab never shows, since a plain view has no data to refresh in the first place.

Limitations to mention: Materialized views support a restricted SQL subset (no non-deterministic functions like CURRENT_TIMESTAMP(), limited join types) — check current BigQuery docs for the exact supported surface. They also cost storage, unlike a plain view, and refresh has some latency (near-real-time, not instantaneous).

7. Partitioning

The problem this solves: attendance_monthly has 1,200 rows in this small demo — trivial to scan in full every time. But imagine this same table after 5 years across 10,000 employees: millions of rows. A query asking about just June 2025 shouldn't have to scan every other month too. PARTITION BY splits a table into physical chunks by a date column, so a query filtering on that column can skip irrelevant chunks entirely.

```
CREATE TABLE `hr_analytics.attendance_monthly_partitioned`
PARTITION BY month_date AS
SELECT *
FROM `hr_analytics.attendance_monthly`
```

Seeing the benefit: compare the estimated bytes scanned for the same filter, against the unpartitioned table versus the partitioned copy, using --dry_run (or the BigQuery console's live query-validator estimate, which shows the same number as you type).

```
-- Query A: against the ORIGINAL, unpartitioned table
bq query --use_legacy_sql=false --dry_run <<'EOF'
SELECT AVG(days_present)
FROM `hr_analytics.attendance_monthly`
WHERE month_date = '2025-06-01'
EOF

-- Query B: against the PARTITIONED copy
bq query --use_legacy_sql=false --dry_run <<'EOF'
SELECT AVG(days_present)
FROM `hr_analytics.attendance_monthly_partitioned`
WHERE month_date = '2025-06-01'
EOF
```

Check it in the console: hr_analytics dataset → attendance_monthly_partitioned → Details tab → a Partition section confirms partitioning by month_date, and Table storage shows per-partition row counts — visual proof the table really is split into monthly chunks, not just labeled as one.

At this table's small size, the byte-count difference will be modest — the real-world payoff of partitioning only becomes dramatic at the millions-of-rows scale this demo dataset is deliberately too small to simulate. Worth saying explicitly to a class: the *mechanism* being demonstrated here is exactly what saves enormous cost at real production scale, even though the numbers on screen today look unremarkable.

8. Clustering

The problem this solves: partitioning narrows a query down to one month. But within that month, if we only care about one department, we're still scanning every department's rows for that month. CLUSTER BY sorts data *within* each partition by the given column(s) — a query filtering on that column can then skip blocks of data within a partition too.

```
CREATE TABLE `hr_analytics.attendance_monthly_partitioned_clustered`
PARTITION BY month_date
CLUSTER BY department_id AS
SELECT *
FROM `hr_analytics.attendance_monthly`
```

```
-- Compare a department-and-month-filtered query, partitioned-only vs. partitioned+clustered
SELECT AVG(performance_rating)
FROM `hr_analytics.attendance_monthly_partitioned`
WHERE month_date = '2025-06-01' AND department_id = 101

SELECT AVG(performance_rating)
FROM `hr_analytics.attendance_monthly_partitioned_clustered`
WHERE month_date = '2025-06-01' AND department_id = 101
```

Check it in the console: attendance_monthly_partitioned_clustered → Details tab shows both a Partition section AND a Clustering section listing department_id — the one place in the console where both optimizations are visible on the same table at once.

Caveat: clustering's benefit doesn't always show up clearly in a --dry_run estimate the way partition pruning does — for a ground-truth comparison, run both queries for real and compare actual Bytes billed in the completed job's details, not just the dry-run estimate.

9. Cleanup

Once the demonstration is complete, remove everything created in this guide — the materialized view, the view, both partitioned/clustered table copies, the three base tables, and the dataset itself.

```
bq rm -f -t YOUR_PROJECT_ID:hr_analytics.monthly_attendance_summary_mv
bq rm -f -t YOUR_PROJECT_ID:hr_analytics.department_summary_view
bq rm -f -t YOUR_PROJECT_ID:hr_analytics.attendance_monthly_partitioned
bq rm -f -t YOUR_PROJECT_ID:hr_analytics.attendance_monthly_partitioned_clustered
bq rm -f -r -d YOUR_PROJECT_ID:hr_analytics
```

Check it in the console: BigQuery > SQL workspace > Explorer → the hr_analytics dataset should no longer be listed under your project at all, confirming every table, view, and materialized view created in this guide was removed along with it.

This guide's companion notebook: everything above has a fully executable, self-contained Jupyter/Colab notebook counterpart using the Austin Bikeshare public dataset, following the identical concept sequence (queries → joins → views → materialized views → partitioning → clustering) with live console screenshots-equivalent guidance at every step.